This is a massive step forward in architectural design. Moving from a single hardcoded hallway to a true **World Grid System** with multiple object types completely changes how your engine needs to think.

Since we are just brainstorming, let's map out how these objects will interact, how the grid keeps track of everything, and how to set yourself up for that mini-map canvas later.

1. The Global "Grid System" Object

Right now, you have an activeHallway variable. In this new world structure, you need a master coordinator—let's call it the WorldGrid or DungeonMap.

Instead of tracking one floating hallway, this structural object will hold everything:

- **mainHallways**: An array of exactly 7 slots.
- **interconnectingHallways**: A dynamic collection (or map) of the shorter tunnels created by the player.
- **user**: The player's active position, but with an upgrade! The user can no longer just have a forwardOffset. They now need to track *which* hallway they are currently inside, and their exact coordinates within that specific hallway.

2. Refining the Creation Engines

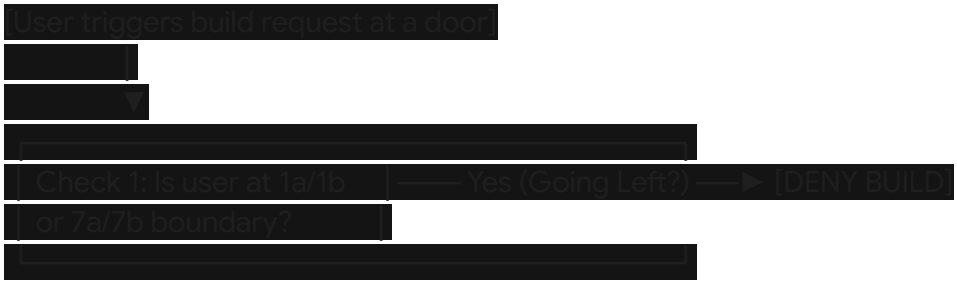
createMainHallway() (The Automated Factory)

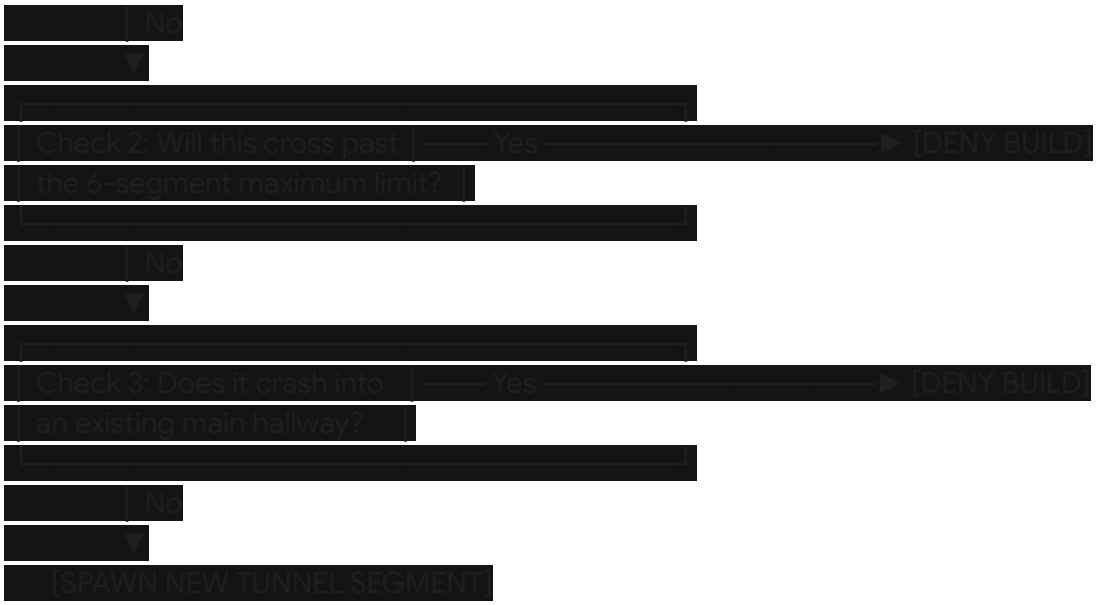
You are exactly right—the user never calls this. This function runs automatically 7 times when the webpage first loads. It builds the 7 permanent structural pillars of your maze, calculates their random offsets relative to the Start Line (**S**), and ensures that the "Overlap Rule" is mathematically met. Once generated, these 7 hallways are locked in place.

createInterconnectingHallway() (The User Action)

This is the interactive engine. When the user types a command (like "/build forward" or "/open side door") while standing at a door node, this function fires.

Because it’s a brainstorm, let's look at the hidden math checks this function must perform before it allowed to place a tunnel:





3. The 3 Object Architectures (Brainstorming Data Models)

To keep your code modular and readable for your YouTube viewers, we can organize our thinking around three clean blueprints:

A. Main Hallway Object

- **ID:** 1 through 7 (to know its sequence in the matrix).
- **Shift Distance:** The random offset number from **S**.
- **Fixed Assets:** Labels (1a/1b), side-door colors, and the 5 specific door nodes.

B. Interconnecting Hallway Object

- **Grid Coordinates:** Where does it start (e.g., *Started at Hallway 2, Door Node 3*) and which direction is it heading?
- **Length:** A fixed scale length exactly equal to the gap distance between two main hallways.
- **State/Type:** It needs to know what is at its far end: 0 (Open Void), 1 (Aligned with a real Main Hallway Door), or 2 (Hitting a Dead-End Wall).

C. The User Object (The Upgraded Navigator)

- **currentHallwayType**: Is the player walking in a "main" hallway or an "interconnecting" tunnel?
- **hallwayId**: The specific ID of the hallway they are sweating in.
- **positionOffset**: Where they are along that specific tunnel's timeline.

4. Setting Up for the Mini-Map Canvas

Thinking ahead about the floating bird's eye view mini-map is brilliant. Because you are separating the **rendering sub-systems** from the **data models**, drawing the mini-map later will be remarkably straightforward.

To draw the mini-map, you will create a secondary, smaller canvas overlaying the screen. Its rendering loop will simply do this:

1. Look at the WorldGrid object.
2. Read the shift distances of the 7 main hallways and draw them as 7 horizontal lines on the mini-map.
3. Read the array of interconnecting hallways and draw them as tiny vertical connecting lines.
4. Read the user's current hallway ID and position offset, convert that mathematical coordinate into a 2D pixel coordinate, and draw a **bright red circle**.

This keeps your world completely data-driven. The 3D canvas and the 2D mini-map canvas will just be two different lenses looking at the exact same data structures.

How does this structural split feel for organizing the next phase of the project?